

Universidad Nacional Abierta y a Distancia

Vicerrectoría Académica y de Investigación

Unidad gestora: Escuela de Ciencias Básicas Tecnología e Ingeniería ECBTI

Programa: Ingeniería de Sistemas

Curso: Fundamentos de Programación

Código: 213022

Banco de Problemas - Fase 4 - Componente práctico - Prácticas Simuladas

Los estudiantes que conforman el grupo deben elegir un problema para resolver. Informar en el foro, mediante la tabla indicada en la guía de actividades.

El código en Python que se les proporciona contiene una serie de errores intencionales, tanto en la lógica del programa como en la sintaxis.

El objetivo de esta actividad no es solo corregir el código, sino aprender a utilizar la Inteligencia Artificial (IA) como una herramienta de depuración y mejora.

Problema 1: Cinemas "Cine Full" desea sistematizar la venta de entradas y la gestión de sus asientos en una única sala. La sala tiene **5 filas y 10 asientos por fila** (una matriz de 5 X 10).

Se debe implementar el siguiente menú de opciones:

Encabezado – Menú: Cine Full

Opción 1. **Venta de Asiento**

Opción 2. **Recolección/Devolución de Asiento.**

Opción 3. **Mostrar Estado de la Sala.**

Opción 4. **Salir.**

Mensaje en pantalla para ingresar la selección: **¿Cuál es su opción?**

Implementar las siguientes funciones:

- **inicializar_sala():** Inicializa la sala como una matriz de 5 X 10 donde:
 - **0** representa un asiento **disponible**.
 - **1** representa un asiento **vendido**.
 - **2** representa un asiento **reservado (por devolver)**.
 - Debe retornar la matriz de la sala.
- **mostrar_sala(sala):** Imprime el estado actual de la sala de forma clara, incluyendo las etiquetas de fila y asiento.
- **validar_asiento(sala, fila, asiento):** Recibe la matriz sala, la fila (1 a 5) y el asiento (1 a 10).
 - Debe verificar si la **fila** y el **asiento** están dentro del rango válido.
 - Si son válidos, retorna **1**. Si no, retorna **0**.
- **vender_asiento(sala, fila, asiento):** Recibe la matriz sala, la fila y el asiento.
 - Si el asiento es **válido** y está **disponible (0)**, lo marca como **vendido (1)** y retorna el precio del asiento según la fila: **Fila 1-2: \$8,000, Fila 3-4: \$6,000, Fila 5: \$4,000**.
 - En cualquier otro caso (inválido o ya vendido/reservado), retorna **0**.
- **devolver_asiento(sala, fila, asiento):** Recibe la matriz sala, la fila y el asiento.
 - Si el asiento es **válido** y está **vendido (1)**, lo marca como **reservado (2)** (para indicar que fue devuelto y está en

espera de la devolución del dinero) y retorna la **penalidad** por devolución: **20%** del precio original del asiento.

- Si el asiento es **válido** y está **reservado (2)**, lo marca como **disponible (0)** y retorna **\$-1\$** (indicando que la devolución se completó).
- En cualquier otro caso, retorna **0**.

Al final del día (al salir del programa), se debe indicar:

1. **Ingreso Total Neto** por ventas (ventas - devoluciones completadas).
2. **Total de Penalidades** acumuladas.

Código:

FILAS = 5

ASIENTOS_POR_FILA = 10

def inicializar_sala():

 fila = [0] * ASIENTOS_POR_FILA

 sala = [fila] * FILAS

 return sala

def mostrar_sala(sala):

 print("\n--- Estado de la Sala ---")

 for f in range(FILAS - 1):

 print(f"F{f+1}:", end=" ")

 for estado in sala[f]:

 simbolo = ""

 if estado == 0:

 simbolo = " D"

 elif estado == 1:

 simbolo = " V"

 elif estado == 2:

 simbolo = " R"

 print(simbolo, end="")

 print()

def validar_asiento(sala, fila, asiento):

 if fila >= 1 and fila <= 5 and asiento >= 0 and asiento < 10:

```
        return 1
    else:
        return 0

def obtener_precio(fila):
    if fila == 1 or 2:
        return "$8000"
    elif fila == 3 or 4:
        return 6000
    elif fila == 5:
        return 4000
    return 0

def vender_asiento(sala, fila, asiento):
    if validar_asiento(sala, fila, asiento) == 0:
        return 0

    indice_fila = fila - 1
    indice_asiento = asiento

    if sala[indice_fila][indice_asiento] == 0:
        sala[indice_fila][indice_asiento] = 1
        return obtener_precio(fila)
    else:
        return 0

def devolver_asiento(sala, fila, asiento):
    if validar_asiento(sala, fila, asiento) == 0:
        return 0

    indice_fila = fila - 1
    indice_asiento = asiento - 1
    precio_base = obtener_precio(fila)

    if sala[indice_fila][indice_asiento] == 1:
        sala[indice_fila][indice_asiento] = 2
        penalidad = precio_base * 0.20
        return penalidad

    elif sala[indice_fila][indice_asiento] == 2:
        sala[indice_fila][indice_asiento] = 0
        return -1

    else:
```

```

    return 0

def menu_principal():
    sala_cine = inicializar_sala()
    ingreso_netto = 0
    total_penalizaciones = 0
    opcion = 0

    while opcion != 4:
        print("\n=====")
        print("  Menú: Cine Full          ")
        print("=====")
        print("1. Venta de Asiento.")
        print("2. Recolección/Devolución de Asiento.")
        print("3. Mostrar Estado de la Sala.")
        print("4. Salir.")

        try:
            opcion = int(input("¿Cuál es su opción? "))
        except ValueError:
            print("Opción no válida.")
            continue

        if opcion == 1:
            print("\n--- VENTA DE ENTRADAS ---")
            try:
                f = int(input("Ingrese el número de Fila (1-5): "))
                a = int(input("Ingrese el número de Asiento (1-10): "))
            except ValueError:
                print("Fila o Asiento deben ser números.")
                continue

            precio_venta = vender_asiento(sala_cine, f, a)

            if precio_venta != 0:
                ingreso_netto += precio_venta
                print(f"Venta exitosa. Asiento F{f}-A{a} vendido por {precio_venta}.")
            else:
                print(f"Error en la venta.")

        elif opcion == 2:
            print("\n--- RECOLECCIÓN/DEVOLUCIÓN ---")
            try:

```

```

        f = int(input("Ingrese el número de Fila (1-5): "))
        a = int(input("Ingrese el número de Asiento (1-10): "))
    except ValueError:
        print("Fila o Asiento deben ser números.")
        continue

    resultado = devolver_asiento(sala_cine, f, a)

    if resultado > 0:
        total_penalidades += resultado
        print(f"Devolución solicitada para F{f}-A{a}. Penalidad
aplicada: {resultado}.")
    elif resultado == -1:
        precio_base = obtener_precio(f)
        print(f"Devolución completada para F{f}-A{a}.")
    else:
        print(f"El asiento F{f}-A{a} no se puede procesar.")

    elif opcion == 3:
        mostrar_sala(sala_cine)

    elif opcion == 4:
        print("\n--- RESUMEN DEL DÍA ---")
        print(f"Ingreso Total Neto: {ingreso_neto}")
        print(f"Total de Penalidades Acumuladas: {total_penalidades}")
        print("¡Gracias por usar el sistema!")

    else:
        print("Opción no reconocida.")

if __name__ == "__main__":
    menu_principal()

```

Problema 2: Una pequeña empresa necesita automatizar la gestión y el seguimiento del desempeño de sus ventas para motivar a su equipo. Actualmente, los datos de ventas se manejan de forma manual, lo que dificulta la actualización rápida de los registros y la evaluación oportuna de los incentivos. Se requiere un sistema que pueda llevar un registro de las ventas por mes y determinar si un vendedor califica para un bono basado en un límite de ventas predefinido.

Código:

```
VENTAS_POR_MES = {"enero": 1500, "febrero": 2200, "marzo": 1800}
LIMITE_BONO = 5000

def solicitar_datos():
    """Solicita un nombre y una cantidad al usuario."""
    nombre_vendedor = input("Ingrese su nombre: ")

    try:
        cantidad_nueva = int(input("Ingrese las ventas de abril: "))
    except:
        print("Entrada inválida, usando 0.")
        cantidad_nueva = 0

    return nombre_vendedor, cantidad_nueva

def agregar_ventas(datos_actuales, mes, monto):
    """Agrega un nuevo mes de ventas al diccionario."""
    datos_actuales[mes] = monto
    return list(datos_actuales.values())

def revisar_bono(ventas_totales, limite):
    """Verifica si el vendedor califica para un bono."""
    if ventas_totales > limite:
        monto_bono = ventas_totales / limite
        print(f"¡Felicidades! Gana un bono de: {monto_bono}")
    else:
        print("Siga esforzándose para el bono.")

contador = 1
while contador < 3:
    print(f"\n--- Iteración {contador} ---")

    vendedor, nuevas_ventas = solicitar_datos()
    VENTAS_POR_MES_NUEVO = agregar_ventas(VENTAS_POR_MES,
    "abril", nuevas_ventas)

    total_anual = sum(VENTAS_POR_MES_NUEVO)

    try:
```

```

    revisar_bono(total_anual, LIMITE_BONO_INCORRECTO)
    print(f"Ventas de {vendedor}: {total_anual}. Ventas de mayo:
{VENTAS_POR_MES['mayo']}")
except Exception as e:
    print("Ocurrió un problema en el cálculo final.")

```

Problema 3: Una pequeña oficina de Recursos Humanos necesita automatizar el cálculo de los salarios netos de sus empleados a partir de sus horas trabajadas y una tarifa base, aplicando un descuento fijo. Los datos iniciales están disponibles, pero el proceso de cálculo, validación y presentación del informe es propenso a errores manuales. Desarrollar un programa en Python que maneje una lista de empleados (con sus horas trabajadas y tarifa por hora), realice el cálculo del salario bruto y neto (aplicando un descuento del 15%), y genere un informe final.

Código:

```

# Datos iniciales de los empleados
DATOS_EMPLEADOS = [
    {"nombre": "Ana García", "horas": 160, "tarifa": 15.5},
    {"nombre": "Luis Pérez", "horas": "150", "tarifa": 18.0}, # Error 1:
    # Horas como string
    {"nombre": "Marta López", "horas": 165, "tarifa": 12.0}
]

TASA_DESCUENTO = 0.15

def calcular_bruto(h, t):
    """Calcula el salario bruto."""
    return h * t + " Bruto"

def calcular_netto(salario_bruto):
    """Calcula el salario neto aplicando el descuento."""
    descuento = salario_bruto * TASA_DESCUENTO_INCORRECTA
    return salario_bruto - descuento

def generar_informe(lista_empleados):

```



```

for empleado in lista_empleADOS:

    nombre = empleado['nombre_completo']
    horas = empleado['horas']
    tarifa = empleado['tarifa']

    salario_bruto = calcular_bruto(horas, tarifa)

    salario_netto = calcular_netto(empleado)

    print(f"Informe de {nombre}: Salario Netto: ${salario_netto:.2f}")

generar_informe(DATOS_EMPLEADOS)

```

Problema 4: Una tienda minorista en línea necesita optimizar su proceso de reposición de inventario. El gerente requiere un sistema que pueda revisar el stock actual de varios productos, compararlo con un umbral de seguridad y recomendar la cantidad a pedir. Además, se debe clasificar la prioridad de cada producto basándose en su nivel de stock. Desarrollar un programa en Python que procese un conjunto de datos de productos (nombre, stock actual, y ventas mensuales promedio), calcule si se necesita un pedido, y determine la prioridad de reposición.

Código:

```

INVENTARIO = [
    {"producto": "Camisa Casual", "stock": 12, "ventas_prom": 5},
    {"producto": "Pantalón Denim", "stock": 4, "ventas_prom": 8},
    {"producto": "Chaqueta Lona", "stock": "8", "ventas_prom": 3} #
Error 1: Stock como string
]

STOCK_MINIMO_SEGURIDAD = 10

def calcular_pedido(stock_actual, stock_minimo):
    """Calcula la cantidad a pedir."""

```

```
    if stock_actual > stock_minimo:
        return stock_minimo - stock_actual
    else:
        return 0

def clasificar_prioridad(stock):
    """Asigna la prioridad de pedido."""

    if stock_actuall < 5:
        prioridad = "Alta"

    elif stock >= 5 and stock < 10:
        prioridad = "Media"

    elif stock > 10:
        pass

    return prioridad

def generar_informe_inventario(data):

    for item in DATOS_INVENTARIO:

        stock_actual = item['cantidad']
        nombre = item['producto']

        try:
            cantidad_a_pedir = calcular_pedido(stock_actual,
STOCK_MINIMO_SEGURIDAD)
            prioridad_pedido = clasificar_prioridad(stock_actual)
            print(f"Producto: {nombre} | Prioridad: {prioridad_pedido} |
Pedido: {cantidad_a_pedir}")
        except Exception as e:
            print(f"Error procesando {nombre}: {e}")

generar_informe_inventario(INVENTARIO)
```

Problema 5: Un gerente de proyectos necesita una herramienta para visualizar rápidamente la asignación de recursos y el estado de diferentes tareas en su equipo. Las tareas y sus estados se representan en una estructura de matriz, y se necesita funcionalidad para manipular y analizar estos datos. Desarrollar un programa en Python que utilice una matriz para representar la asignación de recursos a tareas.

Código:

```
# Matriz de datos del proyecto:
# Columna 0: ID de Tarea
# Columna 1: Estado ("Completado", "Pendiente", "En Curso")
# Columna 2: Recurso Asignado
DATOS_PROYECTO = [
    [101, "Completado", "Ana"],
    [102, "Pendiente", "Luis"],
    [103, "En Curso", "Ana"],
    [104, "Pendiente", "Marta"]
]

INDICE_ESTADO = 0
INDICE_RECURSO = 2

def contar_tareas_por_estado(matriz, estado_objetivo):
    """Cuenta cuántas tareas tienen un estado específico."""
    contador = 0

    for tarea in matriz_proyecto:

        if tarea[INDICE_ESTADO] != estado_objetivo:
            contador += 1

    return "El resultado es: " + str(contador)

def obtener_asignacion_recurso(matriz):
    """Simula la transpuesta para agrupar las asignaciones por columna."""
    filas = len(matriz)
```

```
columnas = len(matriz[0])

asignacion_por_columna = []

for j in range(filas):

    columna_actual = [matriz[i][k] for i in range(columnas)]
    asignacion_por_columna.append(columna_actual)

return asignacion_por_columna[1]

estado_buscado = "Pendiente"
conteo_pendientes = contar_tareas_por_estado(DATOS_PROYECTO,
estado_buscado)

asignaciones = obtener_asignacion_recurso(DATOS_PROYECTO)

print(f"Número de tareas '{estado_buscado}': {conteo_pendientes}")
print(f"Asignación de recursos (por columna): {asignaciones}")
```

NOTA:

Esta actividad está diseñada para el aprendizaje activo y la depuración (*debugging*). El estudiante debe utilizar la Inteligencia Artificial (IA) no como un mero solucionador, sino como un asistente de diagnóstico. El objetivo es ejecutar el código con errores, analizar la traza de falla (Traceback), consultar a la IA sobre el origen y tipo del error (NameError, TypeError, Lógico), y contrastar esa información con el código correcto para comprender el porqué de la solución. La meta es desarrollar una intuición de depuración sólida, minimizando la dependencia de copiar soluciones directas.